Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026, 7:59:59 AM MYT

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick “No” or write “None” – do not leave it blank. Submit as a single PDF document named “SDC2026_TeamName_TechDeclaration.pdf” together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	NylonDaBest
Project / Solution Name	SupplySwipe (Wakil)
Name (Leader)	Kayla Elisheva Siwi
Email & Phone (Leader)	kaykayelisheva@gmail.com / +62 812-8000-6835
University Affiliation(s)	Monash University Malaysia
Live Prototype URL	https://supply-swipe--shariqnaumann.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/kaylaelishevaa/supply-swipe (public)

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

The user opens the Vite + React web app, deployed on Replit, on a phone. Every read and write goes through one module (lib/api.ts), which calls the Python FastAPI backend and falls back to built-in synthetic data when the backend can't be reached; the hosted demo runs that way, with a rule-based engine in the browser generating each negotiation (code picks every price, no model is called). The backend runs the negotiation engine: two Wakil agents exchange at most 20 structured messages, and code checks each offer against the sender's own sealed limit. It also enforces staff approval limits, seals a deal only when both owners approve, and keeps an append-only activity log. Data is held in memory or, optionally, synced to Supabase Postgres. Optionally, the engine asks Claude Haiku 5.5 (Anthropic API) to draft each message as JSON; without it, a rule-based fallback produces the negotiation.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
TypeScript 5.9.3	Frontend language	Apache-2.0
Python 3	Backend language	PSF
Vite 7.3.7	Frontend build and dev server	MIT
React 19.1.0 / React DOM 19.1.0	UI library	MIT
Tailwind CSS 3.4.19, PostCSS 8.5	Styling	MIT
Recharts 2.15.4	Price convergence chart	MIT
three.js 0.160.0 (r160, jsDelivr CDN)	Optional 3D Wakil agent; 2D fallback without WebGL	MIT
Plus Jakarta Sans, JetBrains Mono (Google Fonts CSS)	Typography	SIL OFL 1.1
Next.js 14.2.35; ESLint 8.57 / eslint-config-next 14.2.35	Archived Next.js tooling; not used by the Vite build	MIT
FastAPI ≥0.110 (tested 0.136.1)	Backend API	MIT
Uvicorn ≥0.28 (tested 0.47.0)	ASGI server	BSD-3-Clause
Pydantic ≥2.6 (tested 2.13.4)	Request and response models	MIT
supabase-py ≥2.3	Optional database client	MIT
python-dotenv ≥1.0.1, httpx ≥0.27	Config loading; test client	BSD-3-Clause
Anthropic Python SDK (optional)	Calls Claude from the negotiation engine	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Anthropic Messages API (Claude Haiku 5.5)	Optional: drafts each Wakil message in the backend negotiation engine. Off unless the anthropic package is installed and ANTHROPIC_API_KEY is set; the hosted demo does not call it	Paid (team's own account)
Supabase (Postgres API)	Optional: stores businesses, limits, negotiations, deals, approvals and the activity log	Free (team's own account)
jsDelivr CDN	Serves three.js for the 3D agent	Free
Google Fonts	Fonts, downloaded at build time by next/font	Free

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit (Replit Deployments)	Hosts the Vite + React frontend using pnpm workspace build/serve commands. The optional FastAPI backend is not deployed.	Replit Core (sponsor-provided)
Supabase	Optional Postgres database (backend/schema.sql)	Free plan

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Claude Haiku 5.5 (claude-haiku-5-5)	Anthropic API	Pre-trained, called through the API; optional, with a rule-based fallback	Anthropic Commercial Terms of Service

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
SupplySwipe demo data (businesses, requests, listings, negotiation transcript, invoice)	lib/mock.ts, backend/app/database.py, backend/schema.sql (this repo)	Synthetic (team-created)	Apache-2.0	N
DOSM PriceCatcher (format reference only; no data used)	https://data.gov.my	Public	CC BY 4.0	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude Code (Anthropic), model Claude Opus 5.5, with parallel sub-agents	Scaffolding the Next.js app and writing the frontend screens and components from the team's design handoff; later, reviewing the code, fixing bugs and updating the docs	Major
Claude (claude.ai design artifacts)	Hi-fi HTML design references (not shipped)	Moderate
Replit Agent	Vite migration, debugging, pitch deck and submission-document preparation	Major

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Team members reviewed every AI-generated change before committing it. The backend has an automated test suite (backend/test_api.py, 13 tests covering the rule checks, sealed limits, approvals and the activity log). The frontend is checked with TypeScript, Playwright and a production build, and every screen was run through the guided /demo. The team can explain and justify each component.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Replit Core (platform)	Hosts the live prototype with Replit Deployments
RunPod	US$25 credit	Not used: the project has no GPU work

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Replit hosts the live prototype that judges use. The app is a standard Vite + React build with no Replit-specific runtime code, so another web host could replace it, but Replit is the required platform and the one we deploy on.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

No user data is collected. The prototype uses only team-created synthetic businesses, people, prices and messages. In the hosted demo, the demo state (deal, approvals, log, Wakil look) stays in the visitor's browser (localStorage). When the optional backend runs, the same synthetic data is held in memory or in the team's Supabase project. When the optional language model is on, the backend sends the Anthropic API one side's synthetic negotiation context: its own sealed limit, public prices and the messages so far. No API keys are in the repository; they live in a gitignored .env. A real deployment would follow Malaysia's PDPA principles: purpose-limited collection, access only by the owning business, encryption, set retention, and access and correction for owners.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

The language model can return invalid JSON or an offer outside the sender's sealed limit. Code checks the price and message text against the sender's own limit, and a rule-based move replaces anything that fails or errors. The model's reason and headline fields are not yet checked for sealed values, so the next step is for code to pick every price and the model to write only the wording. The engine depends on one provider (Anthropic) but runs fully without it. There is no real sign-in yet: side and role come from the request, so sealed limits are enforced in the app, not per user. The hosted demo uses the rule-based engine in the browser, where code picks every price and no model is called.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team’s knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



Kayla Elisheva Siwi, Team Leader

Date: 25 October 2026